Kick Start 2018 - Round H

Analysis: Big Buttons

Big Buttons: Analysis

Small dataset

Since we have 2 choices at every step, there are $2^{\mathbf{N}}$ possible strings of length **N**. We can generate all of these strings in $O(2^{\mathbf{N}})$ time using a naive recursion. For each string, we need to check if any of the **P** strings is its prefix. A simple implementation of the above would run in $O(2^{\mathbf{N}} \times \mathbf{P} \times \mathbf{N})$ time for each case. Since $\mathbf{N} \leq 10$ in this dataset, this is fast enough.

Large dataset

In this dataset, since **N** can be as large as 50, it is not feasible to generate all $2^{\mathbf{N}}$ strings. The main idea in solving the Large dataset is to find the number of invalid strings of length **N** and subtract it from the total number of strings ($2^{\mathbf{N}}$).

If a forbidden prefix has length L, we have $2^{(\mathbf{N} - L)}$ invalid strings of length **N** with that prefix. Also, if we have two strings A and B, and A is a prefix of B, then all strings that have B as their prefix will also have A as their prefix. So we can remove all forbidden prefixes that themselves begin with a different forbidden prefix to avoid overcounting the sum of invalid strings.

We can now easily count the sum of invalid strings by finding the number of invalid strings for each remaining forbidden prefix. We can remove redundant forbidden prefixes in $O(\mathbf{P} \times \mathbf{P} \times \mathbf{N})$ time, and find the counts of strings with the remaining forbidden prefixes in $O(\mathbf{P} \times \mathbf{N})$ time, so the overall running time is $O(\mathbf{P}^2\mathbf{N})$. This is fast enough to solve the Large data set, but can you see how to improve the first step to $O(\mathbf{P} \times \mathbf{N})$ time by using a trie?

Kick Start 2018 - Round H

Analysis: Let Me Count The Ways

Let Me Count The Ways: Analysis

Small dataset

We can solve the Small dataset using dynamic programming. Let *f(x, y, z)* (*x* and *y* are non-negative integers, while *z* is 0 or 1) be the number of ways to arrange a seating order for *x* non-newlywed people and *y* newlywed couples (thus the total is *x* + 2*y* people and seats), such that:

- There is no newlywed couple for which the two members are seated next to each other, and
- If *z* = 1, then one of the *x* non-newlywed people must not sit in the first remaining unassigned seat.

The base case of this function occurs when there are no people left (i.e. *x* = *y* = 0), then *f(x, y, z)* = 1. Otherwise, *f(x, y, 0)* can be defined recursively as follows:

- We can put one of the *x* non-newlywed people in the first seat. Therefore, we need to assign the remaining *x* - 1 non-newlywed people and *y* newlywed couples. This contributes *x* × *f(x - 1, y, 0)* to the total number of ways captured by *f(x, y, 0)*.
- We can put one of the member of the *y* newlywed couples in the first seat. The other member of this couple can be considered as an additional non-newlywed person. Therefore, we need to assign the remaining *x* + 1 non-newlywed people and *y* - 1 newlywed couples. However, note that this additional person cannot be seated next to his/her newlywed partner (i.e. cannot be seated on the first unassigned seat after his/her newlywed partner sit down). Therefore, this contributes 2 × *y* × *f(x + 1, y - 1, 1)* to the total number of ways captured by*f(x, y, 0)*.

Therefore, *f(x, y, 0)* = *x* × *f(x - 1, y, 0)* + 2 × *y* × *f(x + 1, y - 1, 1)*. The cases for *f(x, y, 1)* are similar, except that we can't put one of the *x* non-newlywed people in the first seat. Therefore, *f(x, y, 1)* = (*x* - 1) × *f(x - 1, y, 0)* + 2 × *y* × *f(x + 1, y - 1, 1)*.

This dynamic programming solution runs in O(**N** × **M**) space and time.

Large dataset

We can solve the Large dataset using the inclusion-exclusion principle. Since **M** can be up to 100000, it is infeasible to iterate all possible subset of the newlywed couples. However, we can observe that the number of ways to arrange a seating order such that the newlywed couple P sits adjacently (i.e. the two members of a newlywed couple P sit next to each other) is the same as the number of ways to arrange a seating order such that the newlywed couple Q sits adjacently. In general, for a fixed number *k* of newlywed couples, regardless of which particular couples we choose, the number of seating orders in which all of those *k* newlywed couples have their two members adjacent is the same.

Therefore, we can define a function *g(k)* as the number of ways to arrange a seating order such that for each of the first *k* newlywed couples, the two members of a couple are sitting next to each other. *g(k)* can be computed by multiplying the following:

- The number of ways these newlywed couples sit. Since the members of each newlywed couple must sit next to each other, we can treat each newlywed couple as if it were one

person. Therefore, there are 2**N** - *k* seats for *k* couples. Since the couples are not identical and the order of the couples matters, there are C(2**N** - *k*, *k*) × *k*! ways, where C(n, k) is the number of ways of picking k unordered outcomes from n possibilities (i.e. "n choose k").

- The number of ways of ordering two people in each newlywed couple, which is 2^k.
- The number of ways in which the remaining people can sit, which is (2(**N** - *k*))!.

Therefore, *g(k)* = C(2**N** - *k*, *k*) × *k*! × 2^k × (2(**N** - *k*))!

Finally, we should not forget that *g(k)* counts the number of ways to arrange a seating order such that the first *k* newlywed couples sit adjacently. To consider all subset of newlywed couples of size *k*, we need to multiply *g(k)* by C(**M**, *k*).

Putting all the pieces together, the answer we are looking for is Σ $(-1)^k$ × *g(k)* × C(**M**, *k*) for all 0 ≤ *k* ≤ **M**. By precomputing the value of powers of two, factorials, and their modular inverses, this answer can be computed in O(**N** + **M**) space and time.

Kick Start 2018 - Round H

Analysis: Mural

Mural: Analysis

We can observe that we will have painted ceil(**N**/2) sections in the end, and all these sections would form a contiguous subarray of the input array. Since painting and destroying is done alternatively, it might not be possible to paint any subarray of our choice. Our objective is to find the maximum subarray sum among the set of "paintable" subarrays.

Small dataset

An intuitive approach would rely on Dynamic Programming and try to define a DP state that could encapsulate the state of the painted and the destroyed sections at any point in time. Note that painted section is contiguous, and the destroyed sections are prefixes and suffixes of the input array.
Hence, we can define *f(i, j, l, r)* as the maximum possible achievable score if *i* and *j* are the lengths of the destroyed prefix and suffix, respectively; whereas *l* and *r* denote the left and the right boundaries of the painted subarray. A recurrence can easily be derived by considering at most four further possibilities: we have two ways to extend the mural (by painting the section either to the left or to the right of the already painted boundary), and two ways to extend the destroyed part (either the prefix or the suffix).

Note that it would seem that there are O($\mathbf{N}^4$) different valid states in the above approach, but that is not the case since the sum of lengths of painted and destroyed parts is always the same. We can get rid of the index of the right boundary of the painted subarray (i.e. variable *r*), as it can be implicitly derived from the variables *i* and *j*.

The overall complexity of this approach is O($\mathbf{N}^3$) and that will suffice for the Small dataset.

Large dataset

The solution to the Large dataset relies on an interesting observation that all possible contiguous subarrays of length ceil(**N**/2) are "paintable". If we can prove this fact, we can simply do an O(**N**) rolling window approach over all such subarrays and output the maximum possible sum.

Let's think of an intuitive way to prove this. Say, if we paint the *i*-th section on the first day, what could be the smallest possible index of the left boundary of the mural in the worst case? To achieve the smallest possible index, we will always extend the boundary on the left side; and in the worst case the flood can always extend the prefix, allowing us to paint only the indices after index *ceil(i/2)*(inclusive). And similarly, there would be an upper limit on the maximum possible index of the right boundary.
This means that given the desirable left boundary of the mural, we can figure out the "central point" from which we would begin painting. Now, irrespective of the sequence of destructions, we can always meet the desirable left boundary by always extending our subarray to the left whenever a section on the left is destroyed. Similar arguments can be applied to the right boundary.